Studio Nx

백엔드 238 — 트랙 API

2026-10-18

Backend(FastAPI/Python) 238 — 트랙 API (목록 · 만들기 · 조회 · 고치기 · 지우기)

항목	내용
파일	`studionx/backend/app/api/v1/tracks.py`
작업	S1-09 기본 악기·이펙트: 트랙 경로 다섯 개와 공통 오류 모양 (PRD 02 FR-251, FR-252, FR-253)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	237 플러그인 API · 239 악기 API

1. 하는 일

트랙을 다루는 경로 다섯 개를 엽니다. 이 파일 하나가 트랙과 그 안의 악기·이펙트·연결선 경로를 모두 담고 있습니다(239~241). 등록은 `router.py` 에서 **앞 경로 없이**(`prefix` 없이) 붙여, 주소가 `/projects/{id}/tracks` 와 `/tracks/{id}` 가 되게 했습니다.

경로	하는 일	상태 코드
`GET /projects/{project_id}/tracks`	프로젝트의 트랙 목록	200, 401, 404
`POST /projects/{project_id}/tracks`	트랙 만들기	201, 400, 401, 403, 404
`GET /tracks/{track_id}`	트랙 한 건	200, 401, 404
`PATCH /tracks/{track_id}`	보낸 칸만 고치기	200, 400, 401, 403, 404
`DELETE /tracks/{track_id}`	트랙 지우기	204, 401, 403, 404

2. 언제 쓰나

쓰는 곳	
화면의 트랙 추가 단추	`POST` 로 트랙을 만들고 헤더를 그립니다 (FR-253)
프로젝트 열기	`GET /projects/{id}/tracks` 로 트랙 목록과 각 트랙의 상태를 읽습니다
트랙 헤더 믹서	이름·색·볼륨·팬·뮤트·솔로를 `PATCH` 한 번으로 보냅니다
트랙 지우기	`DELETE` 로 트랙과 그 안의 악기·이펙트·연결선을 함께 지웁니다

3. 코드

공통 오류 모양

```
router = APIRouter(tags=["tracks"])

NOT_FOUND_404 = {404: {"model": ErrorResponse}}
WRITE_RESPONSES = {
    400: {"model": ErrorResponse},
    401: {"model": ErrorResponse},
    403: {"model": ErrorResponse},
```

```
    404: {"model": ErrorResponse},
}
```

목록과 만들기

```
@router.get("/projects/{project_id}/tracks", response_model=TrackListResponse, responses={...})
def list_tracks(project_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    items = track_service.list_tracks(db, current_user, project_id)
    return TrackListResponse(
        items=[TrackResponse.model_validate(t) for t in items], total=len(items)
    )


@router.post("/projects/{project_id}/tracks", response_model=TrackResponse,
             status_code=status.HTTP_201_CREATED, responses=WRITE_RESPONSES)
def create_track(project_id: Annotated[int, Path(ge=1)], req: TrackCreateRequest,
                 db: Db, current_user: CurrentUser):
    track = track_service.create_track(
        db, current_user, project_id,
        name=req.name, track_type=req.track_type, color=req.color, volume_db=req.volume_db,
        pan=req.pan, is_muted=req.is_muted, is_solo=req.is_solo, is_enabled=req.is_enabled,
        routing_mode=req.routing_mode, order_index=req.order_index, parent_track_id=req.parent_track_id,
    )
    return TrackResponse.model_validate(track)
```

조회 · 고치기 · 지우기

```
@router.get("/tracks/{track_id}", response_model=TrackResponse, responses={...})
def get_track(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    track = track_service.require_track(db, current_user, track_id)
    return TrackResponse.model_validate(track)


@router.patch("/tracks/{track_id}", response_model=TrackResponse, responses=WRITE_RESPONSES)
def update_track(track_id: Annotated[int, Path(ge=1)], req: TrackUpdateRequest,
                 db: Db, current_user: CurrentUser):
    data = req.model_dump(exclude_unset=True)
    track = track_service.update_track(db, current_user, track_id, **data)
    return TrackResponse.model_validate(track)


@router.delete("/tracks/{track_id}", status_code=status.HTTP_204_NO_CONTENT, responses={...})
def delete_track(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    track_service.delete_track(db, current_user, track_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`WRITE_RESPONSES` 를 만들어 재사용	쓰기 경로 넷이 같은 오류 목록(400·401·403·404)을 씁니다. 한 곳만 고치면 모두 바뀝니다
`{**NOT_FOUND_404}`	사전을 펼쳐 넣습니다. 404 만 필요한 경로에 씁니다
`status_code=status.HTTP_201_CREATED`	만들기 성공은 201 입니다. 화면이 "새로 만들어졌다" 를 구분합니다

식	뜻
`status.HTTP_204_NO_CONTENT` + `Response(...)`	지우기 성공은 본문 없는 204 입니다. `Response` 를 돌려주면 본문이 나가지 않습니다
`req.model_dump(exclude_unset=True)`	요청에 **실제로 들어온** 칸만 사전으로 꺼냅니다. 안 보낸 칸은 빠져 서비스가 건드리지 않습니다
`**data`	사전을 이름 붙은 인자로 풀어 넘깁니다. 서비스의 `**kwargs` 가 받습니다
`total=len(items)`	목록을 한 번 읽고 개수를 셉니다. 트랙 목록은 작아 별도 `COUNT` 쿼리가 필요 없습니다

4. 왜 이렇게

선택	이유
트랙 전 경로를 한 파일에	주소가 트랙을 중심으로 이어집니다(`/tracks/{id}/instruments`). 파일을 나누면 공통 상수를 여러 곳에 두게 됩니다
`prefix` 없이 등록	목록·만들기는 프로젝트 아래(`/projects/{id}/tracks`), 나머지는 트랙 아래(`/tracks/{id}`)라 한 prefix 로 묶을 수 없습니다
204 로 지우기	지운 뒤 돌려줄 내용이 없습니다. 화면은 성공 여부만 알면 됩니다
서비스에 권한을 모두 맡김	라우터에 권한 검사가 섞이면 경로마다 빠뜨릴 수 있습니다. 상태 코드만 정합니다 (233)
403 과 404 를 함께 문서에	남의 프로젝트 트랙은 404, 내 프로젝트지만 VIEWER 면 403 입니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 목록이 `order_index`, `track_id` 순이고 `total` 이 개수와 같음
- 만들기 201과 기본값, 없는 프로젝트 404, VIEWER 403
- 한 건 조회·고치기·지우기(204, 지운 뒤 404)
- 남의 프로젝트 트랙 조회·목록은 404
- 8개 경로가 실제로 등록되었는지 (`test_track_routes_registered`: 트랙 2, 악기 2, 이펙트 2, 연결선 2)